

JiT 结构突破方案：超越 FFN

实验结论回顾

在 JiTMoE baseline 上测试了六类 FFN 改造方案（Timestep-Adaptive Gate、Flow-Evolved Gate、Spatial-Adaptive Value、Frequency-Split、Progressive Refinement、Multi-Step Integrated），结果：

- 除 Frequency-Split 外，所有方案与 baseline SwiGLU 的 FID **几乎完全重合**
- Frequency-Split **略差于** baseline（因 hidden dim 被拆分导致单支容量下降）

核心判断：FFN 不是 JiT 模型的性能瓶颈。

为什么 FFN 改造无效

1. **Timestep 信息已被 AdaLN 充分注入。** AdaLN 在 LayerNorm 处做 scale/shift，FFN 看到的输入本身已经是 timestep-conditioned 的。FFN 内部再加 timestep 调制是重复注入，无边际收益。
2. **FFN 的职责是 per-token channel mixing，天然应该 time-agnostic。** 不同 timestep 下需要的差异化行为已经由输入分布的不同隐式编码。SwiGLU 作为 channel mixer 已经足够强。
3. **空间频率操作不应在 pointwise FFN 里做。** Attention 负责空间交互，FFN 负责通道变换。在 FFN 里插入 depthwise conv / 频率分裂侵入了 attention 的职责范围。

瓶颈在哪

从实验结果倒推，结合 MDTv2、FiTv2、DiffusionBlocks、Flow-based Diffusion 等文献的经验，瓶颈集中在：

- Attention 的空间建模能力（尤其是 JiT 的 token 数少、patch 维度高的设定下）
- 层间信息流的拓扑结构（plain stack 缺少 skip connection）
- 条件注入的参数效率（标准 AdaLN 占参数过多，挤压了可用于增加深度的 budget）

方案 A：Timestep-Aware Attention

动机

JiT 的 attention 目前对所有 timestep 使用完全相同的行为——相同的 QK 投影、相同的 softmax temperature、相同的位置偏置强度。但 flow matching 的两阶段性质 [R2] 意味着 attention 在不同 timestep 下应该有截然不同的聚合模式：

- **Navigation 阶段**（高噪声，早期 t）：模型需要从几乎纯噪声中建立全局 layout。此时 attention 应该**广泛聚合**——每个 token 需要看到尽可能多的其他 token，形成全局一致性。对应行为：较低的 softmax temperature（更平坦的 attention 分布）、较弱的位置衰减。
- **Refinement 阶段**（低噪声，晚期 t）：全局结构已确定，模型需要精修局部细节。此时 attention 应该**局部聚焦**——每个 token 主要关注空间邻域。对应行为：较高的 softmax temperature（更尖锐的 attention 分布）、较强的位置衰减。

这个差异目前被完全忽略了。AdaLN 只调制 FFN 前后的 LayerNorm，不直接改变 attention 的聚合行为本身。

设计 A1: Timestep-Adaptive Attention Temperature

最简单的改动——让 softmax temperature 随 timestep 变化：

```
# 标准 attention
A = softmax(QK^T / sqrt(d_k))

# 改为
 $\tau(t) = \tau_{\text{base}} \cdot \exp(W_{\tau} \cdot t_{\text{emb}} + b_{\tau})$     # 从 timestep 预测温度
A = softmax(QK^T / (sqrt(d_k) *  $\tau(t)$ ))
```

其中 $W_{\tau} \in \mathbb{R}^{d_t \rightarrow 1}$ ，参数增量 $\approx d_t + 1$ ，可忽略。

τ 大 $\rightarrow$ attention 更平坦 $\rightarrow$ 全局聚合 (navigation)

τ 小 $\rightarrow$ attention 更尖锐 $\rightarrow$ 局部聚焦 (refinement)

为什么不直接用一个可学习标量：因为最优温度依赖于当前的噪声水平，而噪声水平在同一个 batch 内不同样本不同。需要从 t_{emb} per-sample 地预测。

设计 A2: Timestep-Modulated Position Bias

在 attention score 上加一个 timestep-controlled 的距离衰减：

```
# 计算 token i 和 token j 的空间距离 (2D grid)
d_ij = ||(h_i, w_i) - (h_j, w_j)||

# 从 timestep 生成衰减强度
 $\lambda(t) = \text{softplus}(W_{\lambda} \cdot t_{\text{emb}} + b_{\lambda})$ 

# Attention score
A_ij = Q_i K_j^T / sqrt(d_k) -  $\lambda(t) \cdot d_{ij}$ 
```

λ 大 $\rightarrow$ 远距离 token 被强烈抑制 $\rightarrow$ 局部 attention (refinement)

λ 小 $\rightarrow$ 远距离抑制弱 $\rightarrow$ 全局 attention (navigation)

这类似于 ALiBi [R7] 的 position bias，但衰减强度不是固定的，而是由 timestep 动态控制。

参数增量： $d_t + 1$ ，可忽略。

设计 A3: Per-Head Timestep Specialization

让不同的 attention head 专注不同的 timestep 范围：

```
# 对每个 head h, 生成一个 timestep-dependent 的 scaling factor
 $s_h(t) = \text{sigmoid}(W_s \cdot t_{\text{emb}} + b_s)$     #  $W_s \in \mathbb{R}^{d_t \times n_{\text{heads}}}$ 

# 每个 head 的输出乘以对应的 scaling
head_h_out =  $s_h(t)[h] \cdot \text{Attention}_h(Q, K, V)$ 
```

这让某些 head 在高噪声下活跃（负责 navigation），另一些在低噪声下活跃（负责 refinement），类似于 MoE 但在 head 级别。

参数增量： $d_t \times n_heads + n_heads$ ，仍然很小。

三个子方案的组合

A1、A2、A3 是正交的，可以叠加：

- **A1 单独**：最简单，建议第一个跑
- **A1 + A2**：温度 + 距离衰减，提供 stronger 的 attention 形状控制
- **A1 + A2 + A3**：完整版，head specialization + attention shape control

参数预算

组件	参数增量	占 baseline 比例
A1 温度预测	$d_t + 1 \approx$ 几百	< 0.01%
A2 距离衰减	$d_t + 1 \approx$ 几百	< 0.01%
A3 Head scaling	$d_t \times n_heads \approx$ 几千	< 0.1%
总计	< 10K	< 0.1%

与现有工作的关系

- **DiffiT** [R8] 提出了 time-dependent self-attention，但它是在 DiT backbone 上做的，具体做法是为不同 timestep range 学习不同的 attention bias。本方案更轻量（不需要 per-range 的独立参数）。
- **Attention Scale in FiTv2** [R9]：FiTv2 在分辨率外推时用了 attention scale factor $s = \max(1.0, \sqrt{\log(H_{\text{test}}W_{\text{test}} / H_{\text{train}}W_{\text{train}})})$ ，但这个 scale 只跟分辨率有关，不跟 timestep 有关。本方案将类似思想推广到 timestep 维度。
- **Flow-based Diffusion 两阶段分析** [R2]：该论文明确指出 navigation 和 refinement 阶段的 attention pattern 应该不同，但没有给出具体的 architecture 改进方案。本方案直接回应了这个 gap。

为什么可能 work

1. 几乎零参数增量，不会引入正则化 / 过拟合问题
2. 直接改变 attention 的信息聚合行为，而不是改变 FFN 的通道变换（已证明无效）
3. 与 flow matching 的两阶段结构高度对齐——navigation 需要 global attention，refinement 需要 local attention
4. 对现有代码侵入性极小——只需改 attention 的 score 计算，不改 block 拓扑

方案 B：层间拓扑重构

动机

JiT / DiT 使用 plain Transformer stack：所有层串行，无 skip connection。但多项研究表明层间信息流的拓扑结构对 diffusion model 至关重要：

- **MDTv2** [R10]: 加入 U-Net style long shortcuts (encoder 浅层 $\rightarrow$ 深层 concat) + dense input shortcuts (原始输入 $\rightarrow$ 每个 decoder block), FID 从 43.82 降到 35.67 (S/2 规模, 600k steps)
- **FiTv2** [R9]: 通过 AdaLN-LoRA 压缩条件注入参数 (从 $6d^2$ 降到 $1.75d^2$), 省出 budget 增加层数 (从 28 层到 36 层), FID 显著改善
- **DiffusionBlocks** [R11]: 证明不同层可以 specialize 到不同噪声范围, 层间无需全局梯度传播

这些证据说明: 相比于改单个 FFN 的内部结构, 改层间的信息传递方式收益大得多。

设计 B1: U-Net Style Long Shortcuts

将 JiTMoE 的 N 个 block 分为前半和后半, 加入对称 skip connection:

```
# 前半 (encoder)
for i in range(N // 2):
    x = block_i(x)
    skip_features[i] = x

# 后半 (decoder)
for i in range(N // 2, N):
    mirror = N - 1 - i
    x = concat(x, skip_features[mirror]) # channel concat
    x = proj_i(x)                       # 投影回原维度
    x = block_i(x)
```

参数增量: 每个 decoder block 多一个 $\text{Linear}(2d, d)$, 共 $N/2$ 个。对于 $d=768, N=12$ 的 B 规模模型, 增量 = $6 \times 768 \times 1536 \approx 7\text{M}$, 约为 baseline 的 5%。

可以通过缩减每层的 AdaLN 参数来抵消 (见 B3)。

设计 B2: Dense Input Shortcuts

将原始 noisy input 接到每个 block 的输入 (MDTv2 验证有效):

```
# x_input = 原始 noisy token embedding (进入第一个 block 之前)
for i in range(N):
    x = concat(x, x_input)
    x = proj_i(x)          # Linear(2d, d)
    x = block_i(x)
```

为什么这对 flow matching 有效: 在 flow matching 中, 模型预测的是 velocity $v = \epsilon - x$, 即"从当前 noisy state 到 clean image 的方向"。每一层都需要记住"当前 noisy state 长什么样"才能准确计算 velocity。Dense input shortcuts 让每层都直接访问这个信息, 而不需要通过很多层的残差连接间接传递。

设计 B3: AdaLN-LoRA 参数压缩

标准 AdaLN 中, 每个 block 需要一个 $\text{Linear}(d, 6d)$ 来生成 6 个 scale/shift 参数。对于 N 个 block, 总参数 = $N \times 6d^2$ 。

FiTv2 的做法: 一个 shared global AdaLN ($\text{Linear}(d, 6d)$) + 每个 block 一个 LoRA (两个小矩阵 $\text{Linear}(d, r)$ 和 $\text{Linear}(r, 6d)$):

```
S_i = W_global · (c + t) + W2_i · W1_i · (c + t)

# W_global ∈ ℝ^{6d × d}      - 共享
# W1_i ∈ ℝ^{r × d}          - per-block
# W2_i ∈ ℝ^{6d × r}          - per-block
```

取 $r = d/4$ 时，每个 block 的 AdaLN 参数从 $6d^2$ 降到 $1.75d^2$ （含共享部分的均摊）。

省出的参数用来增加层数：例如从 12 层增加到 15 层，保持总参数不变。FiTv2 的实验表明更深但每层更瘦的模型效果更好。

三个子方案的组合建议

组合	说明	预期效果
B3 单独	只压缩 AdaLN，增加层数	中等提升，最安全
B1 + B3	Long shortcuts + 更多层	较大提升，MDTv2 已验证
B1 + B2 + B3	完整拓扑重构	最大提升，但实现复杂度也最高

参考文献

- [R9] Wang et al., "FiTv2," 2024. — AdaLN-LoRA + 增加层数的策略。
- [R10] Gao et al., "MDTv2: Masked Diffusion Transformer is a Strong Image Synthesizer," ICCV 2023 / journal 2024. — U-Net shortcuts + dense input shortcuts。
- [R11] DiffusionBlocks, 2025. — 层间独立训练的理论基础。
- [R12] Bao et al., "U-ViT: All are Worth Words," CVPR 2023. — 最早在 ViT-based diffusion 中引入 long skip connections。

方案 C：Patch Embedding 与 Output Head 改造

动机

JiT 的独特之处是 pixel-space x-prediction：网络直接输出 denoised image pixels。Patch size = 16 时，每个 token 维度 = 768 ($16 \times 16 \times 3$)，这意味着：

- **输入端：**每个 token 包含 768 维的 noisy pixel patch，需要先投影到 hidden dim
- **输出端：**每个 token 需要从 hidden dim 解码回 768 维的 clean pixel patch

这两个端点的设计直接决定了“模型预测什么”以及“预测目标有多难”。pMF [R13] 的实验明确表明：x-prediction 成功是因为 clean image 位于低维流形上。那么，如何更好地利用这个低维流形假设？

设计 C1: Timestep-Adaptive Bottleneck

JiT 已经使用了 bottleneck embedding (hidden dim < patch dim)。但 bottleneck 的维度是固定的。考虑到：

- **Navigation 阶段：**目标是多个数据点的混合方向，本身就是低秩的 → 更窄的 bottleneck 就够
- **Refinement 阶段：**目标是单个数据点的精细结构，需要更高维表示 → 更宽的 bottleneck

可以让 bottleneck 的有效维度随 timestep 变化：

```

# Patch embedding
z = W_embed · patch #  $\mathbb{R}^{768} \rightarrow \mathbb{R}^{d_{\text{hidden}}}$ 

# Timestep-dependent masking: 渐进式开启 bottleneck 通道
mask_threshold = sigmoid(W_mask · t_emb) # [B, 1], 标量
channel_mask = sigmoid((arange(d_hidden) / d_hidden - mask_threshold) · temperature)
z = z * channel_mask # 早期只用部分 channel, 后期全部开启

```

直觉：早期只需要少量 channel 描述全局方向，后期需要全部 channel 描述精细结构。这和 progressive growing / coarse-to-fine 的思想一致。

参数增量：仅一个 $\text{Linear}(d_t, 1)$ ，可忽略。

设计 C2: Multi-Scale Patch Embedding

不用单一 patch size，而是用多个 patch size 的 embedding 并融合：

```

# 多尺度 patch embedding
z_16 = PatchEmbed(x, patch_size=16) # 标准：少 token、高维 patch
z_8 = PatchEmbed(x, patch_size=8) # 细粒度：多 token、低维 patch
# z_8 需要 pool 回和 z_16 相同的 token 数

# Timestep-controlled 融合
 $\alpha(t) = \text{sigmoid}(W_\alpha \cdot t_{\text{emb}})$ 
z = (1 -  $\alpha$ ) · z_16 +  $\alpha$  · pool(z_8)

```

Navigation 阶段偏向粗粒度 patch（全局信息），refinement 阶段偏向细粒度 patch（局部细节）。

这个方案实现更复杂，但提供了**输入端的多尺度感受野**，补偿了 attention 在少 token 设定下的空间分辨率不足。

设计 C3: Auxiliary Perceptual Loss

pMF [R13] 已经证明 perceptual loss 对 pixel-space 生成模型非常有效。具体做法：

```

# 标准 v-loss
L_v = ||v_θ(z_t, t) - v||²

# 将 v_θ 转换回 x-prediction
 $\hat{x} = z_t - t \cdot v_\theta(z_t, t)$ 

# Perceptual loss (用预训练 VGG / LPIPS)
L_perceptual = LPIPS( $\hat{x}$ , x_clean)

# 总 loss
L = L_v +  $\lambda \cdot L_{\text{perceptual}}$ 

```

λ 可以随 timestep 变化——refinement 阶段（低噪声）时 perceptual loss 权重更大（因为此时 $\hat{x}$ 已经接近 clean image，perceptual loss 有意义）；navigation 阶段（高噪声）时权重小或为零（因为 $\hat{x}$ 太 noisy，perceptual loss 没有意义）。

注意：这不是结构改动，而是 loss 改动。但它直接改变了“模型学什么”，可能比改结构更有效。

参考文献

- [R4] Li & He, "JiT," 2025. — bottleneck embedding 的原始设计。

- [R13] Lu et al., "One-step Latent-free Image Generation with Pixel Mean Flows (pMF)," 2025. — perceptual loss 在 pixel-space 模型中的有效性。
- [R14] Karras et al., "Analyzing and Improving the Image Quality of StyleGAN (StyleGAN2)," CVPR 2020. — progressive growing 的思想。

三个方案的总结与优先级

方案	核心改动	参数增量	实现难度	预期收益	优先级
A: Attention	attention score 加 timestep 调制	< 0.1%	低	中-高	最高
B: 层间拓扑	skip connections + AdaLN-LoRA + 更多层	≈ 0 (参数重分配)	中	高	高
C: Patch/Loss	自适应 bottleneck + perceptual loss	< 1%	中-高	中	中

推荐实验路径

第一步：跑 A1 (attention temperature) — 最快验证"改 attention 是否有效"

↓

如果 A1 有效 → 叠加 A2 (position bias) → 叠加 A3 (head specialization)

↓

第二步：并行跑 B3 (AdaLN-LoRA + 增加层数) — 独立验证"深度 vs 宽度"

↓

第三步：组合 A + B 的最优配置

↓

第四步：在最优 A+B 配置上加 C3 (perceptual loss)

这条路径的逻辑是：先验证最便宜、最低风险的改动（A1），再逐步叠加更重的改动。每一步都可以独立评估收益。

完整参考文献

- [R1] Lipman et al., "Flow Matching for Generative Modeling," 2023.
- [R2] Liu et al., "From Navigation to Refinement: Two-Stage Nature of Flow-based Diffusion," 2025.
- [R3] Liu et al., "Flow Straight and Fast: Rectified Flow," 2022.
- [R4] Li & He, "Back to Basics: JiT," 2025.
- [R5] LightningDiT related results, 2025.
- [R6] Boned Riera et al., "ODE-ViT," 2025.
- [R7] Press et al., "Train Short, Test Long: Attention with Linear Biases (ALiBi)," ICLR 2022.

- [R8] Hatamizadeh et al., "DiffiT: Diffusion Vision Transformers for Image Generation," ECCV 2024.
- [R9] Wang et al., "FiTv2: Scalable and Improved Flexible Vision Transformer for Diffusion Model," 2024.
- [R10] Gao et al., "MDTv2: Masked Diffusion Transformer is a Strong Image Synthesizer," 2024.
- [R11] DiffusionBlocks, 2025.
- [R12] Bao et al., "U-ViT: All are Worth Words," CVPR 2023.
- [R13] Lu et al., "One-step Latent-free Image Generation with Pixel Mean Flows (pMF)," 2025.
- [R14] Karras et al., "StyleGAN2," CVPR 2020.